



Venus Labs - Deviation Sentinel

Security Assessment

CertiK Assessed on Feb 5th, 2026





Certik Assessed on Feb 5th, 2026

Venus Labs - Deviation Sentinel

The security assessment was prepared by Certik.

Executive Summary

TYPES

Lending

ECOSYSTEM

Binance Smart Chain
(BSC)

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 01/06/2026

Final report published on 02/05/2026

Vulnerability Summary



10

Total Findings

9

Resolved

1

Timelock

0

Partially Resolved

0

Acknowledged

0

Declined



1 Centralization

1 Timelock



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.



0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.



0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.



1 Medium

1 Resolved



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.



2 Minor

2 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.



6 Informational

6 Resolved



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | VENUS LABS - DEVIATION SENTINEL

Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Overview

[PR-24](#)

Dependencies

[Third Party Dependencies](#)

[Recommendations](#)

Findings

[VLD-07 : Centralization Related Risks](#)

[VLD-02 : Solidity Version Reverts On Underflow And Overflow When Contract Assumes Wrapping Behavior](#)

[VLD-03 : Roundabout Logic Can Cause Unexpected Reverts](#)

[VLD-04 : Missing Checks](#)

[VLD-01 : Discussion On Spot Price Usage](#)

[VLD-08 : Missing Error Messages](#)

[VLD-09 : Unused Events](#)

[VLD-10 : Potential Redundant Setting](#)

[VLD-11 : Discussion On Manual Updates To Markets With Deviation Sentinel Enabled](#)

[VLD-12 : Constructor Missing NatSpec Comment](#)

Optimizations

[VLD-06 : Unnecessary Repeated Fetching In Loop](#)

Appendix

Disclaimer

CODEBASE | VENUS LABS - DEVIATION SENTINEL

Repository

<https://github.com/VenusProtocol/venus-periphery>

Commit

Base: [bffb9839a261e174af18af920ce8bc4928eab5dd](#)

Update1: [ab18286dbfa4f966c7adef77b66d7ad22e189b13](#)

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | VENUS LABS - DEVIATION SENTINEL

This audit was conducted for Venus Labs to evaluate the security and correctness of the smart contracts associated with the Venus Labs - Deviation Sentinel project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

OVERVIEW | VENUS LABS - DEVIATION SENTINEL

This audit concerns the changes made in the in scope files outlined in the following PR:

- [PR-24](#)

Note that any centralization risks present in the existing codebase before these PRs were not considered in this audit and only those added in these PRs are addressed in the audit. We recommend all users carefully review the centralization risks, much of which can be found in our previous audits, which can be found here: <https://skynet.certik.com/projects/venus>.

PR-24

This PR introduces the Price Deviation Sentinel system, which monitors price feeds for deviations beyond configured thresholds. It adds core logic to enable/disable certain actions within markets based on these deviation conditions.

DEPENDENCIES | VENUS LABS - DEVIATION SENTINEL

Third Party Dependencies

The protocol is serving as the underlying entity to interact with third party protocols. The third parties that the contracts interact with are:

- ERC20 Tokens
- UniswapV3 Pools
- PancakeswapV3 Pools
- Oracles

The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. Moreover, updates to the state of a project contract that are dependent on the read of the state of external third party contracts may make the project vulnerable to read-only reentrancy. In addition, upgrades of third parties can possibly create severe impacts, such as increasing fees of third parties, migrating to new LP pools, etc.

Recommendations

We recommend constantly monitoring the third parties involved to mitigate any side effects that may occur when unexpected changes are introduced, as well as vetting any third party contracts used to ensure no external calls can be made before updates to its state.

FINDINGS | VENUS LABS - DEVIATION SENTINEL



10

Total Findings

0

Critical

1

Centralization

0

Major

1

Medium

2

Minor

6

Informational

This report has been prepared for Venus Labs to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 10 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
VLD-07	Centralization Related Risks	Centralization	Centralization	1h Timelock
VLD-02	Solidity Version Reverts On Underflow And Overflow When Contract Assumes Wrapping Behavior	Logical Issue	Medium	Resolved
VLD-03	Roundabout Logic Can Cause Unexpected Reverts	Logical Issue	Minor	Resolved
VLD-04	Missing Checks	Logical Issue	Minor	Resolved
VLD-01	Discussion On Spot Price Usage	Logical Issue	Informational	Resolved
VLD-08	Missing Error Messages	Coding Style	Informational	Resolved
VLD-09	Unused Events	Inconsistency	Informational	Resolved
VLD-10	Potential Redundant Setting	Logical Issue	Informational	Resolved
VLD-11	Discussion On Manual Updates To Markets With Deviation Sentinel Enabled	Logical Issue	Informational	Resolved
VLD-12	Constructor Missing NatSpec Comment	Inconsistency	Informational	Resolved

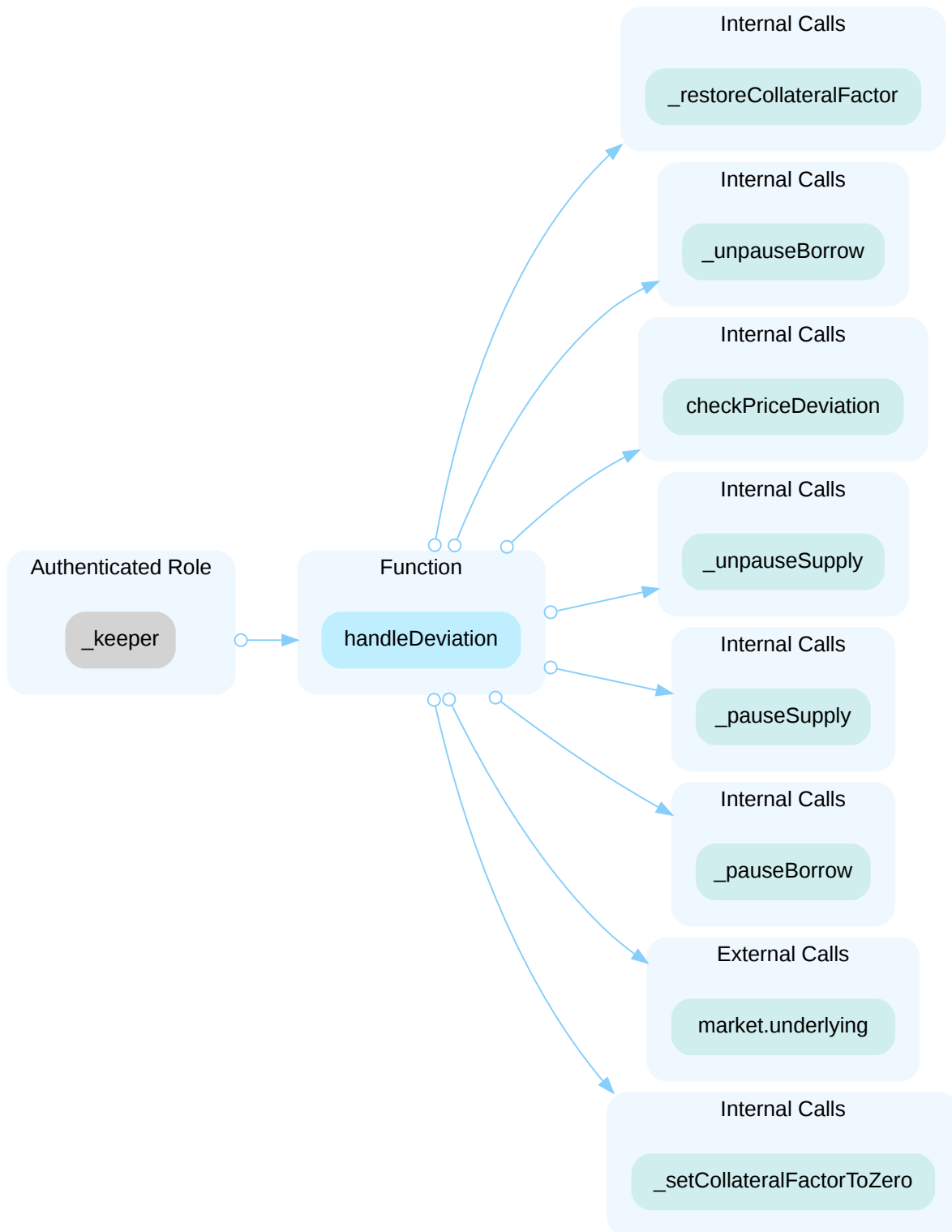
VLD-07 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	● Centralization	DeviationSentinel/DeviationSentinel.sol (Base): 173, 189, 206, 225, 262; DeviationSentinel/Oracles/PancakeSwapOracle.sol (Base): 63; DeviationSentinel/Oracles/SentinelOracle.sol (Base): 52; DeviationSentinel/Oracles/UniswapOracle.sol (Base): 63	● 1h Timelock

Description

DeviationSentinel

In the contract `DeviationSentinel`, the role `_keeper` has authority over the functions shown in the diagram below. Any compromise to the `_keeper` account may allow the hacker to take advantage of this authority and manipulate oracles to allow them to call `handleDeviation()` to either disable or enable functionality unexpectedly.



In the contract `DeviationSentinel`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setTrustedKeeper()`
- `setTokenConfig()`
- `setTokenMonitoringEnabled()`
- `resetMarketState()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set a malicious address as a trusted keeper allowing them to execute the privileges mentioned above.
- Set the deviation threshold for a token higher or lower than expected.
- Set token monitoring to enabled or disabled at unexpected times.
- Reset the market state for a token, when no manual action was taken or at an unexpected time.

PancakeSwapOracle

In the contract `PancakeSwapOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following function:

- `setPoolConfig()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set a malicious pool for the token causing malicious prices to be returned.

UniswapOracle

In the contract `UniswapOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following function:

- `setPoolConfig()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set a malicious pool for the token causing malicious prices to be returned.

SentinelOracle

In the contract `SentinelOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following function:

- `setTokenOracleConfig()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set a malicious oracle for the token causing malicious prices to be returned.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts

with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign ($\frac{2}{3}$, $\frac{3}{5}$) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

I Alleviation

[Venus Labs, 01/09/2026]:

1. Even if keeper account is compromised it won't be a risk to the system because handleDeviation will only update markets if there is an actual deviation.
2. We will transfer the ownership of the contracts to normal timelock and only timelock contracts will have permission to call the functions of the contracts.

VLD-02 | Solidity Version Reverts On Underflow And Overflow When Contract Assumes Wrapping Behavior

Category	Severity	Location	Status
Logical Issue	● Medium	Libraries/FullMath.sol (Base): 1, 37-47	● Resolved

Description

The contract relies on arithmetic operations that assume wrapping behavior on underflow and overflow, as was common in Solidity versions prior to 0.8.0. However, the current implementation uses a Solidity version above 0.8.0, where underflow and overflow in arithmetic operations trigger a revert by default. As a result, certain input values that the library is intended to support, such as `mulDiv(2255, 5, 100)`, will cause a revert instead of producing the intended value. This discrepancy can lead to unexpected transaction failures in valid use cases, potentially affecting contract functionality where large mathematical computations are expected to wrap.

Recommendation

We recommend reviewing all arithmetic operations in the affected library and explicitly use unchecked blocks wherever wrapping on overflow or underflow is intended, to ensure calculations behave as expected under Solidity versions 0.8.0 and above. Carefully identify each instance where modular arithmetic is required and enclose these operations within unchecked blocks, making the code's behavior explicit and preventing unintended transaction reverts. Additionally, update documentation to clearly note where and why unchecked blocks are used for future maintainability.

Alleviation

[CertiK, 01/12/2026]: The client made the recommended changes resolving this finding in commits

- [48ef43416300e175a66bb5405eb25986ba20c2de;](#)
- [0e0a69d0f8953ad1e799067af65372d7745bf53b.](#)

VLD-03 | Roundabout Logic Can Cause Unexpected Reverts

Category	Severity	Location	Status
Logical Issue	Minor	DeviationSentinel/Oracles/PancakeSwapOracle.sol (Base): 110~118; DeviationSentinel/Oracles/UniswapOracle.sol (Base): 107~119, 107~119	Resolved

Description

The current implementation of `_getUniswapV3Price()` and `_getPancakeSwapV3Price()` calculates the target token price using an intermediate variable `targetTokensPerReferenceToken` and multiple divisions/inversions:

```
if (targetIsToken0) {
    targetTokensPerReferenceToken = FullMath.mulDiv(FixedPoint96.Q96 * (10 ** 18),
1, priceX96);
} else {
    targetTokensPerReferenceToken = FullMath.mulDiv(priceX96 * (10 ** 18), 1,
FixedPoint96.Q96);
}

price = FullMath.mulDiv(referencePrice, (10 ** 18), targetTokensPerReferenceToken);
```

The code includes `priceX96 * (10 ** 18)`, which if the price is large enough may exceed the maximum `uint256` and revert due to overflow. Additionally, the operation `FullMath.mulDiv(FixedPoint96.Q96 * (10 ** 18), 1, priceX96)` can round down to zero if the price is large enough, which would cause a revert when dividing by zero to determine the `price`.

The logic can be simplified to increase its readability and remove potential overflow and division by zero errors.

Recommendation

We recommend refactoring the calculation to make it more direct and readable. For example, the price can be equivalently computed as:

```
if (targetIsToken0) {
    price = FullMath.mulDiv(referencePrice, priceX96, FixedPoint96.Q96);
} else {
    price = FullMath.mulDiv(referencePrice, FixedPoint96.Q96, priceX96);
}
```

Alleviation

[CertiK, 01/12/2026]: The client made the recommended changes resolving this finding in commits

- [0e0a69d0f8953ad1e799067af65372d7745bf53b](#);

- 5340dc569c871cd605f97643fa6c847d8d10ed62.

VLD-04 | Missing Checks

Category	Severity	Location	Status
Logical Issue	● Minor	DeviationSentinel/Oracles/PancakeSwapOracle.sol (Base): 90; Deviation Sentinel/Oracles/UniswapOracle.sol (Base): 90	● Resolved

Description

When reading the price from the `v3Pool` it does not check that it is greater than 0. If the pool is uninitialized, it can return a price of zero, which should indicate that an incorrect pool is configured.

Recommendation

We recommend adding the missing checks mentioned above.

Alleviation

[Venus Labs, 01/09/2026]: We will verify the pools liquidity before configuring them.

VLD-01 | Discussion On Spot Price Usage

Category	Severity	Location	Status
Logical Issue	● Informational	DeviationSentinel/Oracles/PancakeSwapOracle.sol (Base): 90; DeviationSentinel/Oracles/UniswapOracle.sol (Base): 90	● Resolved

Description

The protocol relies on a spot price from an on-chain liquidity pool to enforce deviation-based sentinel checks. Spot prices in AMM pools are inherently easy to manipulate, especially in pools with limited liquidity or during low-activity periods. As a result, the spot price may not accurately reflect the fair market value of the underlying asset at the time it is queried.

An attacker can temporarily manipulate the pool price to either:

- Trigger the DeviationSentinel by pushing the price beyond the deviation threshold.
- Avoid triggering the DeviationSentinel by manipulating the price back within bounds at the moment of the check.

Because the check relies on instantaneous price observations, it is susceptible to flash-loan and block-level price shaping, where an attacker controls the ordering and timing of trades to influence the observed price without sustaining long-term cost.

In general only pools with a sufficient amount of liquidity should be used to increase the cost of significant price movements. In such a case, a minimum liquidity requirement could be added.

In addition, instead of using the spot price, a TWAP computed over a sufficiently long window to make manipulation economically infeasible could be implemented. If this is implemented, it should be ensured that the TWAP is fresh.

Recommendation

We recommend clarifying the design choice to utilize the spot price and share how the protocol intends to mitigate these spot price manipulation concerns.

Alleviation

[Venus Labs, 01/09/2026]: We plan to use this kind of config:

- Only pools with substantial liquidity (typically >1M-5M) are configured for monitoring
- Minimum thresholds will be typically 15-25% which requires requires capital expenditure that exceeds the potential profit from triggering/avoiding the sentinel

Also:

- We cannot use TWAP smoothing would create lag, potentially missing the exact moments when intervention is needed

most.

Keepers check prices across multiple blocks. Attacker must manipulate price for duration of keeper check window. So flash loans wont help a attacker.

VLD-08 | Missing Error Messages

Category	Severity	Location	Status
Coding Style	● Informational	Libraries/FullMath.sol (Base): 14, 21	● Resolved

Description

The **require** can be used to check for conditions and throw an exception if the condition is not met. It is better to provide an error or string containing details about the error that will be passed back to the caller.

Recommendation

We recommend adding error messages to the linked **require** statements.

Alleviation

[CertiK, 01/12/2026]: The client made the recommended changes resolving this finding in commit [b2150b7bd355e7814485a9aae331d298d7ac2c94](#).

VLD-09 | Unused Events

Category	Severity	Location	Status
Inconsistency	● Informational	DeviationSentinel/DeviationSentinel.sol (Base): 107~111	● Resolved

Description

The event `CollateralFactorUpdated(address indexed market, uint256 oldCF, uint256 newCF)` is unused and instead the event that includes the `poolId` is used for both core and isolated pools.

Recommendation

We recommend either implementing or removing the unused event. In a addition, if it is removed, we recommend ensuring the comments above the event are consistent with the behavior.

Alleviation

[CertiK, 01/12/2026]: The client made the recommended changes resolving this finding in commit [5588f48e51dcd26710e89c138879e51fd47d1e51](#).

VLD-10 | Potential Redundant Setting

Category	Severity	Location	Status
Logical Issue	 Informational	DeviationSentinel/DeviationSentinel.sol (Base): 177, 195, 213; DeviationSentinel/Oracles/PancakeSwapOracle.sol (Base): 67; DeviationSentinel/Oracles/SentinelOracle.sol (Base): 57; DeviationSentinel/Oracles/UniswapOracle.sol (Base): 67	 Resolved

Description

The following functions permit setting a state variable to its current value. When this occurs, the contract emits an event signaling an update, even though the state remains unchanged. This may mislead users or off-chain systems monitoring events into believing an actual change has taken place.

DeviationSentinel

- `setTrustedKeeper()`
- `setTokenConfig()`
- `setTokenMonitoringEnabled()`

SentinelOracle

- `setTokenOracleConfig()`

PancakeSwapOracle

- `setPoolConfig()`

UniswapOracle

- `setPoolConfig()`

Recommendation

We recommend that the affected setter functions first check whether the new value differs from the current value before updating the state and emitting associated events. If the new value is identical to the current value, the function should return without performing any state changes or event emissions. This will help avoid confusing off-chain systems and users with misleading events that do not correspond to actual contract state changes.

Alleviation

[Venus Labs, 01/09/2026]: We will update these values via VIP so we won't update it to the same values again.

VLD-11 | Discussion On Manual Updates To Markets With Deviation Sentinel Enabled

Category	Severity	Location	Status
Logical Issue	● Informational	DeviationSentinel/DeviationSentinel.sol (Base): 224	● Resolved

Description

There are many fringe cases that may result from manual changes to a market that has the deviation sentinel enabled. However, these depend on how these will be handled procedurally.

Any market changes that affect the collateral factor, borrow pause, or supply pause can cause the markets state to differ from what the DeviationSentinel expects based on its stored state. While there is a function `resetMarketState()` designed to handle this, we would like to ensure the procedural process will ensure consistency and not leave any fringe cases. In particular, can you please share the operational procedure that you will follow when making changes to a market that has the DeviationSentinel token monitoring enabled.

Recommendation

We recommend clarifying the points above.

Alleviation

[CertiK, 01/12/2026]: The client added clarifying comments in commit [ab18286dbfa4f966c7adef77b66d7ad22e189b13](#). However, we leave this finding pending to ensure the following cases have been considered as we are unsure about the definition of "manual intervention" in the market. In particular, according to the added comments we assume this means anytime a markets CF, LT, borrowed paused state, or supply paused state is manually updated. Furthermore, we would like to ensure that it also includes the addition/removal of markets from e-mode pools and the creation of new e-mode pools.

- If a new e-mode pool is created when the protocol is paused due to the sentinel, then the contract can have an inconsistent state.
- If a market is added/removed from an e-mode pool during a pause due to the sentinel, then the contract can have an inconsistent state.

In addition, it should be noted that by always calling `resetMarketState()` before manual intervention in these markets, it can cause markets to be unable to be unpaused via the sentinel and require manual intervention if the market is unexpectedly paused via the sentinel prior to the manual intervention. We would like to ensure this is the intended behavior.

[Venus Labs, 01/28/2026]: If governance makes any e-mode changes (such as creating a pool or adding/removing a market from a pool), the procedure should be followed in this order:

1. Call `setTokenMonitoringEnabled(token, false)` to disable monitoring.
2. Call `resetMarketState(market)` to clear the stored state.

3. Apply all e-mode changes via the VIP.
4. Apply all CF/LT/pause changes through the same VIP.
5. Decide whether to re-enable monitoring.

We must ensure that governance strictly follows this sequence.

[CertiK, 01/28/2026]: We leave this finding so the following comments can be seen. In addition, we sent a comment in the report that there can also be an inconsistent state:

- If a market is relisted/unlisted in the isolated pools during a pause due to the sentinel, then the contract can have an inconsistent state.

[Venus Labs, 02/05/2026]: Yes, we will follow the same process in IL as well.

VLD-12 | Constructor Missing NatSpec Comment

Category	Severity	Location	Status
Inconsistency	● Informational	DeviationSentinel/DeviationSentinel.sol (Base): 144; DeviationSentinel/Oracles/PancakeSwapOracle.sol (Base): 43; DeviationSentinel/Oracles/SentinelOracle.sol (Base): 34; DeviationSentinel/Oracles/UniswapOracle.sol (Base): 43	● Resolved

Description

The `constructor()` lacks the NatSpec annotation `@custom:oz-upgrades-unsafe-allow constructor`. This annotation is used to explicitly inform the OpenZeppelin Upgrades plugin that the use of a constructor is intentional, helping to avoid automated warnings or errors during contract deployment or upgrade processes.

Recommendation

We recommend adding the NatSpec comment.

Alleviation

[CertiK, 01/12/2026]: The client made the recommended changes resolving this finding in commit [583e2d3fdb9ae543b5e8dfb6fece5877aa17f16c](#).

OPTIMIZATIONS | VENUS LABS - DEVIATION SENTINEL

ID	Title	Category	Severity	Status
VLD-06	Unnecessary Repeated Fetching In Loop	Code Optimization	Optimization	● Resolved

VLD-06 | Unnecessary Repeated Fetching In Loop

Category	Severity	Location	Status
Code Optimization	● Optimization	DeviationSentinel/DeviationSentinel.sol (Base): 234, 399, 452	● Resolved

Description

Within multiple for loops that iterate over all e-mode pools, the external call `CORE_POOL_COMPTRROLLER.lastPoolId()` is invoked repeatedly. Since the value returned by `lastPoolId()` does not change during the execution of these functions, making multiple identical external calls is unnecessary. Storing the result in a local variable before entering the loop would eliminate redundant external calls and improve gas efficiency.

In addition, if the `corePoolId` is intended to always be zero and never be updated, then it can be replaced by a constant 0 to save the external call.

Recommendation

We recommend storing the result of `CORE_POOL_COMPTRROLLER.lastPoolId()` in a local variable before the start of each loop, and reference this local variable within the loop, to avoid repeated external calls and improve gas efficiency. In addition, if `corePoolId` is always intended to be zero, to instead reference a constant value of 0.

Alleviation

[CertiK, 01/12/2026]: The client updated the code to cache these values in commit [157c217279e1241fd7db9ec51b85fe8a0e1f7ee5](https://github.com/VenusLabs/DeviationSentinel/commit/157c217279e1241fd7db9ec51b85fe8a0e1f7ee5).

APPENDIX | VENUS LABS - DEVIATION SENTINEL

Audit Scope

VenusProtocol/venus-periphery

DeviationSentinel/Oracles/PancakeSwapOracle.sol

DeviationSentinel/Oracles/SentinelOracle.sol

DeviationSentinel/Oracles/UniswapOracle.sol

DeviationSentinel/DeviationSentinel.sol

Libraries/FullMath.sol

Libraries/FixedPoint96.sol

Interfaces/IComptroller.sol

Interfaces/ICorePoolComptroller.sol

Interfaces/IFlashLoanReceiver.sol

Interfaces/IILComptroller.sol

Interfaces/IPancakeV3Pool.sol

Interfaces/IProtocolShareReserve.sol

Interfaces/IUniswapV3Pool.sol

Interfaces/IVBNB.sol

Interfaces/IVToken.sol

Interfaces/IWBNB.sol

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

